

MISAKA

consensus-native EVM 実行レーンを備えた耐量子 BlockDAG Layer-1

MISAKA プロジェクト

github.com/MISAKA-BTC/misakas ・ エクスプローラ: misakascan.com

ホワイトペーパー v0.1 — June 18, 2026

概要. MISAKA は *rusty-kaspa* のフォークとして構築された、耐量子・PQ-only の Layer-1 である。Kaspa の GHOSTDAG BlockDAG コンセンサスと UTXO 台帳を保ちつつ、コンセンサス表層で古典的に破られる原始関数をすべて置換する。トランザクション認可は secp256k1/Schnorr から ML-DSA-87(FIPS 204, NIST カテゴリ 5) へ移行し、乗算的 MuHash の UTXO アキュムレータは格子加法的な LtHash32_1024 となり、コンセンサス識別子 (block hash・txid・Merkle root・UTXO commitment・parent 参照) 全体を 64 バイトの keyed BLAKE2b-512 ダイジェスト (*Hash64*) に拡張する。Proof-of-Work は二層構成であり、コンセンサス上重要な Layer 0 finalizer は 512 ビット target と比較する。稼働中の Layer 1 は検証高速性のために選ばれた compute-only な BLAKE2b-512 || SHA3-512 タグである。compute-only PoW はもはや ASIC 平等的ではないため、MISAKA は第二の独立資源で深いリオーガを抑止する。すなわち、ステークを置いたバリデータが canonical anchor に attestation を行い、stake-confirmed anchor を脱出する候補フォークは累積 work と累積 stake の両方で canonical chain を上回らねばならない、という *DNS 型確率的ファイナリティ・オーバーレイ* である。この上で、外部ブリッジも L2 シーケンサも用いずに、Ethereum EVM を *L1* コンセンサスの一部として実行する。各 DAG block の EVM parent をその GHOSTDAG selected parent に束縛し、Kaspa 準拠の *mergeset* 遅延受理でトランザクションを実行することで、EVM 結果は block の parents の append-only な関数となり、仮想リオーガは再実行ではなくポインタ切替で済む。本書では暗号・コンセンサス・ファイナリティ・オーバーレイ・EVM レーンとその UTXO ブリッジ・ネイティブコインの発行スケジュール・計画中の Fact Settlement Layer を述べ、全体を通じて仕様保証値と技術目標値を明確に分離する規律を保つ。MISAKA はその DAG エンジンを Kaspa プロジェクトから継承しており、上流のすべての功績は Kaspa に帰する。

Contents

1	はじめに	1
2	予備知識と記法	2
3	システム概要	3
3.1	一つの DAG 上の二レーン	3
3.2	設計目標	3
3.3	非目標	4
4	耐量子暗号コア	4
4.1	脅威モデルと PQ 主張の範囲	4
4.2	ML-DSA-87 によるトランザクション認可	4
4.3	Hash64—64 バイトのコンセンサス識別子	5
4.4	LtHash32—格子 UTXO アキュムレータ	5
4.5	アドレスと単一の標準 script	5
4.6	Mass と DoS ポリシ	5
4.7	secp-free 保証	6

5	コンセンサスと Proof-of-Work	6
5.1	継承された DAG コンセンサス	6
5.2	階層型 proof-of-work	6
5.3	幅チェーンと work 会計	6
5.4	Layer 1 タグの変遷と compute-only の代償	7
5.5	難易度・活性化・再 genesis	8
5.6	伝播安全性	8
6	DNS 確率的ファイナリティ・オーバーレイ	8
6.1	なぜ第二資源か	8
6.2	オーバーレイが加えるもの	8
6.3	全アクティブ・ステーカー attestation	9
6.4	品質ゲート付き StakeScore(BFT-free)	9
6.5	二次元リオーグゲート	9
6.6	長距離境界・ロールアウト・パラメータ	9
6.7	運用上の堅牢化—バリデータとリモート signer	10
7	Selected-Parent EVM 実行レーン	10
7.1	緊張関係と selected-parent 規則	10
7.2	Mergeset 遅延受理 (off-by-one)	10
7.3	Commitment 構造とバージョンゲーティング	11
7.4	EVM 実行環境	11
7.5	Validity と 5 分類 skip セマンティクス	12
7.6	Gas 予算と二段階上限	12
7.7	手数料モデル	13
7.8	段階的スケーラビリティ	13
7.9	PQ-only との整合と secp 隔離	13
7.10	コンセンサス不変条件	14
7.11	性能主張の規律	14
8	UTXO ↔ EVM ブリッジ	14
8.1	供給不変条件	15
8.2	Deposit(二段階)	15
8.3	Withdraw(F002 precompile)	15
9	ネイティブコインと発行	15
10	Fact Settlement Layer(ロードマップ)	16
11	実装とネットワーク状況	17
12	セキュリティ考察	17
13	関連研究	18
14	結論と今後の課題	18

1. はじめに

MISAKA を動機づけるのは二つの圧力である。第一は暗号的圧力である。ほぼすべての UTXO チェーンを守る署名・アキュムレータの原始関数—secp256k1 の ECDSA/Schnorr と離散対数型アキュムレータ—は、Shor のアルゴリズムを走らせる十分大きな量子計算機に対して破られ、

チェーン識別子を支える対称ハッシュは Grover 探索によって原像強度の半分を失う。第二は構造的圧力である。線形チェーンの決済は遅く、Kaspa が先導した BlockDAG モデルは、ブロックの有向非巡回グラフを GHOSTDAG プロトコルで順序付けることにより高ブロックレート時の orphan ペナルティを除去し、最長鎖型の安全性議論を犠牲にせず高い BPS(blocks per second) を維持する。

MISAKA はこれらを、原始関数が古典的に破れたままのチェーンへオーバーレイとして後付けするのではなく、一つの基盤層の上で同時に解くべきだという立場を取る。具体的には、MISAKA は rusty-kaspa の **PQ-only** フォークであり、既存チェーンのアップグレード経路ではなく、Kaspa とも従前の実験的 kaspa-pq の状態ともワイヤ・アドレス・ウォレットいずれの互換性も持たない。独自 genesis を持つ新しいネットワークであり、その上ではすべての非耐量子経路—旧来の ECDSA/Schnorr/multisig 署名、旧来アドレス、pay-to-script-hash(P2SH)—が consensus・mempool・wallet の各層で表現不能にされる。

この PQ 基盤に、決済・プライバシー専用のコインとは一線を画す二つの機構を加える。第一は *DNS 確率的ファイナリティ・オーバーレイ* である。これはブロック生成とは分離された proof-of-stake の確認層であり、攻撃者がハッシュパワーとステーク価値の両方を支配しない限りネットワークが深いリオーグを拒否できるようにする。この二資源確認こそが、検証が高価なメモリ困難ハッシュから、ノードがマイクロ秒で検証できる compute-only ハッシュへと proof-of-work を意図的に変更することを正当化する—同期ボトルネックを除きつつ、純 PoW による履歴の書き換えを安価にはしない。第二は *consensus-native EVM レーン* である。外部ブリッジも別個のシーケンサも用いず、L1 コンセンサスの一部として完全な Ethereum 仮想マシンを実行し、本質的に逐次的な EVM が、再編成を繰り返す DAG と、仮想変更のたびの再実行を強いることなく共存するよう設計されている。

本書は MISAKA のソースツリーに実装・凍結された設計を記述する。記述は意図的に網羅的であり—暗号・コンセンサス・proof-of-work・ファイナリティ・EVM レーン・価値ブリッジ・発行・計画中の Fact Settlement Layer—ある構成要素が稼働コンセンサスではなくロードマップ項目である場合はその旨を明記する。全体を通じて体裁は技術ノートの慣行に従い、性能数値は仕様保証値と技術目標値を分離する明示的な規律 (Section 7.11) に従う。

構成。 Section 2 で記法と BlockDAG の前提を定める。Section 3 で二レーン構成と設計目標・非目標を示す。Section 4 で耐量子暗号コアを規定する。Section 5 でコンセンサスと階層型 proof-of-work を扱う。Section 6 で DNS ファイナリティ・オーバーレイを展開する。Section 7 はプロトコルの中核である selected-parent EVM レーンである。Section 8 で UTXO↔EVM 価値ブリッジを、Section 9 でネイティブコインと発行を、Section 10 で計画中の Fact Settlement Layer を規定する。Sections 11 to 14 で実装状況・セキュリティ考察・関連研究・結論を扱う。

2. 予備知識と記法

バイト列の連結を $\parallel$ 、長さ接頭辞を $\text{len}(\cdot)$ と書く。鍵付きハッシュは、メッセージ m を鍵 k の下で取った BLAKE2b-512 を $\text{BLAKE2b-512}(\text{key} = k, m)$ と書く。MISAKA は鍵スロットをドメイン分離にのみ使い、攻撃者が境界を再整合できる文字列接頭辞には決して用いない。Uin t_w は符号なし w ビット整数型を表す。サイズは特記なき限りバイト単位である。ネイティブコイン 1 MSK は 10^8 sompi、EVM の 1 wei は 10^{-10} sompi である (Section 8)。

BlockDAG と GHOSTDAG。 BlockDAG は頂点をブロックとする有向非巡回グラフ $G = (V, E)$ であり、各ブロックは一つ以上の *parent* を指名し、 E はブロックからその *parent* へ向かう。ブロック B の過去 $\text{past}(B)$ は B から到達可能なブロックの集合、*anticone* はその過去にも未来にも属さないブロックの集合である。GHOSTDAG [1] は各ステップで相互に良く接続された「青い」ブロックの k -クラスタを貪欲に選ぶことで G を線形化する。これはブロックごとの *selected parent* $\text{sp}(B)$ を通じて *selected chain* を誘導し、 $\text{past}(B)$ 上の全順序を定める。 k はネットワーク遅延に合わせて調整され、伝播窓内に生成された正直なブロックがほぼ常に青く

なるようにする。selected chain に沿って累積する proof-of-work が *blue work* であり、tip 選択は *blue work* を最大化する。

Mergeset. chain block B の *mergeset* は

$$\text{mergeset}(B) = \text{sp}(B) \cup (\text{anticone}(\text{sp}(B)) \cap \text{past}(B)), \quad (1)$$

すなわち B がその selected parent に対して「マージ」するブロック群である。本書で繰り返し用いる基本性質 (Section 7) は、selected chain 上で連続する chain block の mergeset が互いに素であり、その和が DAG の過去を被覆することである。これにより mergeset ごとの集計規則は exactly-once になる。

UTXO 台帳とアキュムレータ。 状態は未使用トランザクション出力 (UTXO) の集合である。簡潔な *UTXO commitment* はこの集合を累積し、block header が一定サイズで台帳状態に commit でき、追加・削除が増分的に行えるようにする。上流 Kaspas は 3072 ビットの乗算的 MuHash を用いる。Section 4 でこれを置換する。

確認とファイナリティ。 本書では (i) 累積 work に対する確率的確認 (最長 DAG 型の言明)、(ii) DNS オーバーレイの *stake-confirmed anchor* (Section 6)、(iii) EVM レーンの RPC ファイナリティタグ *latest/ safe/ finalized* (Section 7) を区別する。これらは別個の対象であり、決して混同しない。

3. システム概要

3.1. 一つの DAG 上の二レーン

MISAKA は、単一の GHOSTDAG BlockDAG と単一の PoW を共有しつつ、それ以外は予算・gossip・I/O が分離された二つの実行レーンを提供する (Figure 1)。

- **UTXO レーン**は ML-DSA-87 の PQ-only である。既存の DAG-inclusive acceptance (トランザクションは、その block が仮想 selected chain の accepted 集合に順序付けられた時点で有効になる) を用い、10 BPS で動作する。その規則と性能は設計の他の部分によって不変に保たれる。
- **EVM レーン**は selected-parent chain 上で *mergeset* 遅延受理 (Section 7) により実行される Ethereum 仮想マシンである。secp256k1/ECDSA を独立した署名ドメインとして再導入するが、既定ノードバイナリが secp-free を保つようビルド feature の背後に隔離される。

レーン間の価値移動は、単一のネイティブコイン供給を保存する in-consensus の deposit/withdraw 機構 (Section 8) を通じて行われる。第二の launch 後コンセンサス層である DNS ファイナリティ・オーバーレイ (Section 6) は両レーンと直交して動作し、二資源 (work × stake) 規則で深いリオーグを抑止する。

3.2. 設計目標

1. **今まさに重要な箇所を耐量子化する。** トランザクション認可とコンセンサス識別子を耐量子化し、UTXO アキュムレータを格子ベースにする。トランスポートの秘匿性は ML-KEM ハイブリッドを有効にしない限り耐量子ではないことを明示する。
2. **DAG エンジンを保つ。** GHOSTDAG のブロック生成、blue-work による tip 選択、難易度調整アルゴリズム (DAA)、mempool、短期確認は Kaspas から不変に継承する。
3. **二資源による深いリオーグ耐性。** 確認済み履歴の書き換えに work と stake の双方の支配を要求する PoS ファイナリティ・オーバーレイを加える。これが高速な compute-only PoW を許容可能にする。

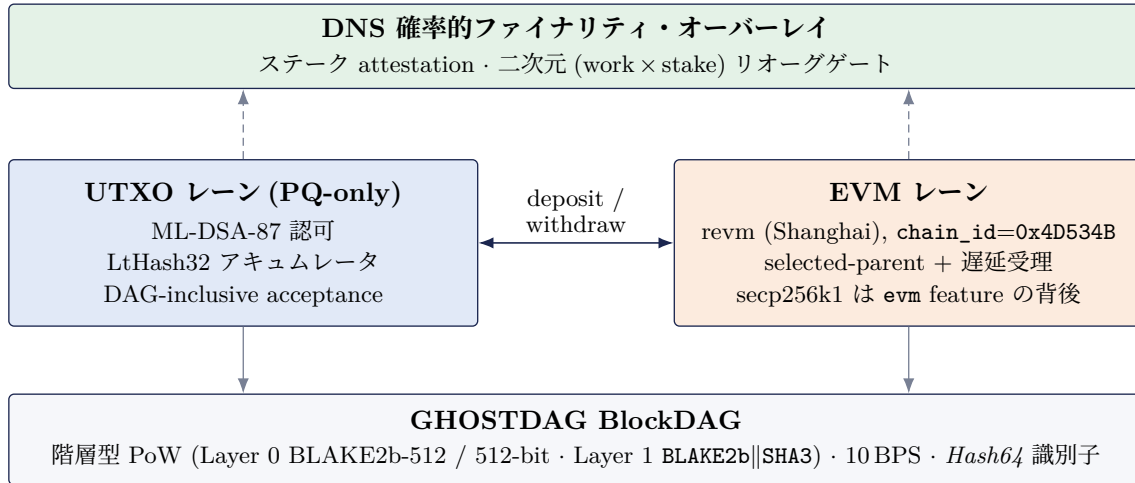


Figure 1. MISAKA の二レーン構成。一つの GHOSTDAG BlockDAG と一つの proof-of-work が PQ-only の UTXO レーンと consensus-native な EVM レーンを担い、in-consensus ブリッジが単一供給を保存し、DNS ファイナリティ・オーバーレイが work $\times$ stake 規則で深いリオーグを抑制する。

4. **L1 コンセンサスとしての EVM。**外部ブリッジも L2 シーケンサも用いずに EVM を実行しつつ、UTXO 台帳と DAG の安定性を不変に保ち、「EVM 結果は block hash に対して不変、リオーグはポインタ切替」を全段階で真に保つ。
5. **誠実な性能会計。**仕様保証値(コンセンサスパラメータから導出可能)と技術目標値(ベンチマーク裏付け)を分離し、単一の hot object の数値を一般スループットとして広告しない。

3.3. 非目標

MISAKA は mainline Kaspas のアドレス・ウォレット・RPC・P2P 互換性を維持しない。secp256k1 UTXO の移行も行わない。「すべての暗号が耐量子」と主張しない(トランスポートは launch 時スコープ外)。ML-DSA multisig や P2SH を launch スコープに含めない。全ノード再実行の EVM について単一数値のスループット王座を主張しない—その目標は研究段階の validity-proof レーン (Section 7) に留保する。EVM レーンは明示的に耐量子ではない。その署名ドメインは secp256k1 であり、量子脅威に対する露出は Ethereum と同等であり、これは利用者へ開示される。

4. 耐量子暗号コア

4.1. 脅威モデルと PQ 主張の範囲

最終的に Shor と Grover のアルゴリズムを大規模に実行する攻撃者を想定する。Shor は離散対数・素因数分解の仮定を破り、secp256k1 署名と MuHash の基礎をなす RSA 型群を無効化する。Grover は対称ハッシュへの原像・衝突探索を平方根的に高速化し、実効強度を半減させる。MISAKA の PQ 主張は精密かつ限定的である。

トランザクション認可は ML-DSA-87 を用いる。PQ コンセンサスモードでは secp256k1 署名は無効化される。コンセンサス識別子は 64 バイトの BLAKE2b-512 ダイジェストである。UTXO アキュムレータは格子ベースである。トランスポート層の秘匿性は ML-KEM ハイブリッド鍵交換を有効にしない限り耐量子ではない (launch 時スコープ外)。「すべての暗号が耐量子」とは主張せず、旧来の Kaspas アドレスは明示的に耐量子ではない。

4.2. ML-DSA-87 によるトランザクション認可

署名は ML-DSA-87(FIPS 204、標準化された Module-Lattice DSA、NIST セキュリティカテゴリ 5) であり、ピン留めした libcrux-ml-dsa($\geq 0.0.9$ 、コンセンサス経路では exact-pin + lockfile 監査)を用いる。パラメータサイズは公開鍵 2592、秘密鍵 4896、署名 4627 バイトであり、script

識別子	型	生成元
Block hash	BlockHash=Hash64	header preimage 上の keyed BLAKE2b-512
Transaction id	Hash64	tx preimage 上の keyed BLAKE2b-512
Merkle root 群	Hash64	hash-Merkle / accepted-id Merkle、64 バイトノード
UTXO commitment	UtxoCommitment64	LtHash32 finalize(Section 4.4)
Pruning point, parents	Hash64	block-hash 参照

Table 1. 64 バイト/512 ビット (keyed BLAKE2b-512) へ拡張したコンセンサス識別子。EVM 内部 root は Ethereum ツール互換のため keccak-256/32 バイトのまま。

上に載る署名要素は `signature || sighash_type = 4628` バイトである。ドメイン分離には context 文字列 `kaspa-pq-v2/tx/mldsa87` を用いる。

署名 transcript は専用の 64 バイト sighash であり、scheme と幅の双方で弱い旧来 32 バイト Schnorr sighash ではない。

$$\text{sighash}_{\text{mldsa87}}(\text{tx}, i, \text{type}) = \text{Hash64}(\text{意味的フィールド} \parallel \text{ドメイン kaspa-pq-v2/sighash/mldsa87} \parallel \text{class tag}), \quad (2)$$

ここで script-class tag は標準 ML-DSA-87 P2PKH クラスを束縛し、構成要素の各ハッシュ (previous outputs · sequences · sig-op counts · outputs · payload) も Hash64 に拡張される。検証側はこの 64 バイトダイジェストを、トランザクション context の下で ML-DSA-87 検証ルーチンへ直接渡す。PQ 経路に `secp256k1::Message` は現れない。署名検証キャッシュは (公開鍵 · 署名 · メッセージ) の 3 つの BLAKE2b-512 ダイジェストをアルゴリズムタグの下でキーにするため、生の鍵 · 署名は保持されない。

4.3. Hash64—64 バイトのコンセンサス識別子

512 ビットの PoW finalizer(Section 5) と拡張済み UTXO commitment があっても、block hash · txid · Merkle root · pruning point · parent 参照を 256 ビットのままにすれば、それらの原像コストは Grover の下で 2^{128} に抑えられ、量子マージンの非対称が再導入される。そこで MISAKA は、コンセンサスから見える識別子全体を 64 バイト/512 ビットとし、該当 preimage 上の keyed BLAKE2b-512 で生成する (*Hash64* 型)。Table 1 に移行対象を挙げる。MISAKA は新しいネットワークであるため、一度きりのワイヤ・格納形式コストを稼働チェーンへ後付けするのではなく前払いする。ただし Ethereum ツール互換のために意図的な例外を設ける。EVM レーンの内部では state/receipts/transactions root は keccak-256/32 バイト (EvmH256) のままとし、EVM 実行に対する L1 コンセンサス commitment のみを Hash64 とする (Section 7)。

4.4. LtHash32—格子 UTXO アキュムレータ

MuHash アキュムレータは 3072 ビット剰余の乗算群であり、その安全性は Shor が破る離散対数型仮定に依拠する。MISAKA はこれを *LtHash*—安全性が格子 (short-integer-solution) 問題に帰着し、追加・削除が逆元による同一の成分ごと演算となる加法的アキュムレータ—で置換する。監査改訂を経て LtHash32_1024(32 ビットレーン 1024 本、4096 バイト状態、 ≈ 170 ビットの量子衝突マージン) を採用し、 ≈ 100 ビットの量子マージンが最弱リンクだった 16 ビットレーン版から引き上げた。アキュムレータは 2^{32} を法とする成分ごと加法の可換群であり、空状態 finalize は 64 バイトの BLAKE2b-512 で、全体で用いる keyed-BLAKE2b 族を共有する。

4.5. アドレスと単一の標準 script

唯一のアドレス版は PubKeyHashM1Dsa87 であり、その payload は検証鍵の keyed BLAKE2b-512、64 バイトである。

$$\text{payload} = \text{BLAKE2b-512}(\text{key} = \text{kaspa-pq-v2/address/mldsa87}, vk).$$

mainnet アドレスは `misaka`、testnet は `misakatest`、devnet は `misakadev` で始まる。唯一の標準 script は ML-DSA-87 の pay-to-public-key-hash である。

OP_DUP OP_BLAKE2B_512 OP_DATA64 <payload64> OP_EQUALVERIFY OP_CHECKSIG_MLDSA87

P2SH は無効である。単に非優先化するのではなく、script 分岐の実行前に拒否し、redeem script を経て旧来署名 opcode が密かに復活しないようにする。旧来署名 opcode(OP_CHECKSIG・OP_CHECKSIGECDSA・OP_CHECKMULTISIG および verify 系) は PQ モードでコンセンサス失敗となる。重要なのは、これらの検査が mempool だけでなくコンセンサス検証器と script エンジンに存在することであり、block を直接投入するマイナーも回避できない。ML-DSA multisig は launch スコープ外である。後に追加する場合も、専用に静的解析されるクラスとし、P2SH の一括再有効化は決して行わない。

4.6. Mass と DoS ポリシ

ML-DSA-87 のオブジェクトは大きいため、署名検証を価格付けし block 計算を有界化するよう mass モデルを再校正する。署名演算 mass は $\text{mass_per_sig_op} = 10,000$ 、最大 script element は 8192 バイト (4628 バイト署名と 2592 バイト鍵が収まる)、script および signature-script の長さ上限は 16,384 バイト (ML-DSA P2PKH の unlock は ≈ 7.3 KB) である。dust 閾値は上流の 148 バイト前提ではなく ML-DSA P2PKH の spend サイズから算定する。計算 mass 上限により block あたり概ね 197 件の ML-DSA トランザクションに制限される。

4.7. secp-free 保証

既定ビルドでは secp256k1 依存が kaspas-consensus とノードバイナリ kaspad の双方から feature gate で除外される (既定 feature は pq-only)。旧来 opcode 実装・旧来ウォレット API・旧来テストは、非ノードツールのみが用いる legacy-secp256k1 feature の背後に置かれる。継続的インテグレーションのガード (scripts/pq-ci-guard.sh) は、依存ツリーの辺が secp256k1 を consensus crate やノードへ再導入するとビルドを hard-fail させ、同ガードは 9 個の本番バイナリを対象とする。EVM レーンの secp256k1 (Section 7) は別個の evm ビルド feature (既定オフ) の背後でのみ再導入される。

5. コンセンサスと Proof-of-Work

5.1. 継承された DAG コンセンサス

ブロック生成・tip 選択・work 順序・難易度は Kaspas から継承し、PQ 移行で変更しない。ブロックは PoW で生成され、GHOSTDAG が tip を選び、work 順序は blue work アキュムレータであり、難易度調整アルゴリズムは blue-work 駆動である。MISAKA は全ネットワークを 10 BPS— $\text{target_time_per_block} = 100 \text{ ms}$ —で動作させ、GHOSTDAG クラスタパラメータ k はネットワーク遅延境界とブロックレートから標準規則 $k = \text{GHOSTDAG-K}(2\lambda D, \delta)$ で算出する (λ はブロックレート、 D は伝播遅延)。

5.2. 階層型 proof-of-work

上流 Kaspas の PoW は三つの関心事を混同している。比較ドメイン (マイナーが解く整数幅)、ASIC 耐性関数 (マイナーが走らせる重い計算)、work 会計幅である。MISAKA はこれらを分離し、各層が独自の hard-fork スケジュールで動けるようにする。PoW は二層構成である (Figure 2)。

$$\begin{aligned}
 \text{L1_tag} &= \text{AsicHardFn}_{\text{algo_id}}(\text{pre_pow_hash}, \text{timestamp}, \text{bits}, \text{nonce}) \\
 \text{pow}_{512} &= \text{BLAKE2b-512}\left(\text{key} = \text{kaspas-pq-pow-v1}, \text{netid} \parallel \text{algo_id} \parallel \text{pre_pow_hash} \parallel \text{timestamp} \right. \\
 &\quad \left. \parallel \text{bits} \parallel \text{nonce} \parallel \text{len}(\text{L1_tag}) \parallel \text{L1_tag} \right) \\
 \text{受理} &\iff \text{Uint512}(\text{pow}_{512}) \leq \text{target}_{512}(\text{bits}).
 \end{aligned} \tag{3}$$

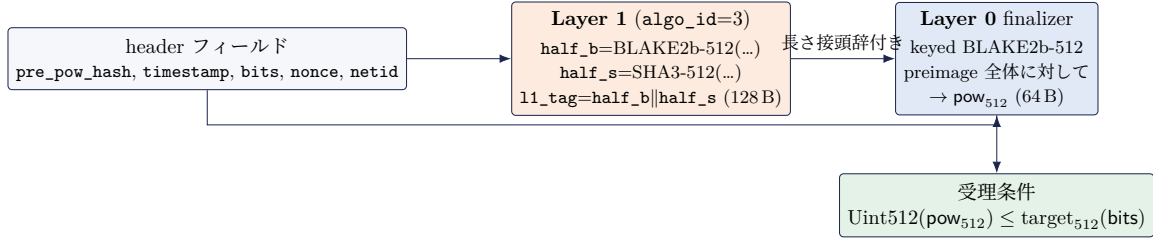


Figure 2. 二層 PoW。compute-only な Layer 1 タグ (BLAKE2b-512 || SHA3-512、algo_id=3) は、コンセンサス上重要な Layer 0 BLAKE2b-512 finalizer へ長さ接頭辞付きで入力され、その 512 ビットダイジェストが 512 ビット target と比較される。Layer 0 は生の header フィールドも再吸収するため、マイナーは SHA3 半分を省略できない。

量	型	幅	備考
bits	コンパクト	32 ビット	header 符号化 target
PoW target / ダイジェスト	Uint512	512 ビット	Layer 0 比較ドメイン
単一ブロック work	Uint576 (BlueWorkType)	576 ビット	$\lfloor 2^{512}/(\text{target} + 1) \rfloor$
DAA 窓演算	Uint640	640 ビット	集計ヘッドルーム

Table 2. PoW 幅チェーン。アキュムレータは到達不能を意図した安全上限として target より 1 ワード広い。

Layer 0—耐量子 finalizer(コンセンサス上重要)。 64 バイトを生成し 512 ビット target と比較する keyed BLAKE2b-512(HAIFA 構造)である。kasper-pq-pow-v1 で鍵付けされ、自己区切りである。preimage が network_id と Layer 1 タグへの明示的な長さ接頭辞を埋め込むため、将来の algo_id 変種が従前の符号化と衝突しない。Layer 0 族は Layer 1 が用いる Keccak/SHA-3 族と意図的に異なるため、一方の族の構造的弱点が同一 PoW の両半分を貫通しない。Layer 0 は凍結されており、追加の hard-fork ADR なしには変更されない。

Layer 1—ASIC 耐性タグ。 header 内の 8 ビット algo_id で識別される。切替は hard cut-off であり、二つの algo_id 値が同一 DAA score で同時に有効になることはなく、混合アルゴリズムの難易度演算は存在しない。

5.3. 幅チェーンと work 会計

幅は work が桁あふれしないように連鎖する (Table 2)。コンパクトな 32 ビット bits フィールドは 512 ビット target に展開され、finalizer は 512 ビットダイジェストを生み、単一ブロック work は 576 ビットアキュムレータに保持される $\text{work} = \lfloor 2^{512}/(\text{target} + 1) \rfloor$ (target より 1 マシンワード広く、 2^{64} 個の最大 work ブロックの窓でも収まる) であり、DAA 窓演算は 640 ビットを用いる。難易度リフト恒等式 $\text{target}_{512} = \text{target}_{256} \ll 256$ はブロック発見確率を厳密に保存し ($\Pr[X_{512} \leq \text{target}_{256} \ll 256] = \text{target}_{256}/2^{256}$)、移行経路ではなく 256-512 ビット写像の文書記録として機能する (MISAKA は新規チェーン)。

5.4. Layer 1 タグの変遷と compute-only の代償

Layer 1 アルゴリズムは三段階を経た。**Phase 1**(algo_id=1) は上流の kHeavyHash 行列を再輸出し、ASIC 耐性の主張はしなかった。**Phase 2**(algo_id=2) は GPU 対 ASIC の差を圧縮するためメモリ困難な Argon2id(16 MiB) タグを出荷した。これは機能したが構造的な代償を露呈した。メモリ困難 PoW は対称的であり、検証者がマイナーのハッシュごとコストの大部分を支払う。最弱メッシュホストでの実測では Argon2id ヘッダ検証 1 回が ≈ 48 ms であり、同期中のノードが CPU の大半を Argon2id 再実行に費やすため、PoW 検証が initial-block-download のボトルネックになる。

Phase 3(algo_id=3)、すなわち稼働中のアルゴリズムは、Argon2id を compute-only タグ

で置換する。

half_b = BLAKE2b-512(key = kasper-pq-11-blake2b-sha3-v1, len(netid) || netid || pre_pow_hash || nonce),
 half_s = SHA3-512(kasper-pq-11-blake2b-sha3-v1 || len(netid) || netid || pre_pow_hash || nonce),
 l1_tag = half_b || half_s (128 バイト).

(4)

128 バイトタグは変更のない Layer 0 finalizer へ入力され、finalizer は preimage 全体 (half_s を含む) に対して二度目の BLAKE2b-512 を混合するため、マイナーは SHA3 半分を省略できない。nonce ごとの work は $2 \times \text{BLAKE2b-512} + 1 \times \text{SHA3-512}$ であり、検証は概ね $10^4 \times$ 高速 ($\approx \mu\text{s}/\text{header}$) になる。

代償は明示的かつ受容されている。compute-only PoW はメモリ困難ではないため、GPU/FPGA/ASIC による加速が可能であり、Phase 3 は kHeavyHash/Argon2id の ASIC 耐性目標を放棄する。これが許容されるのは PoW がもはや唯一の安全柱ではないからである。二次元 DNS ファイナリティ・オーバーレイ (Section 6) がリオーグを stake-confirmed anchor に対して抑止するため、純 PoW 多数派でも確認済み履歴を書き換えられない。本決定は、実運用上の真の苦痛である同期・検証コストを、オーバーレイが既に和らげている PoW 平等性より優先する。二つのハッシュ族の連結は暗号的ヘッジであり、BLAKE2b-512 か SHA3-512 のいずれかが破れても Layer 0 比較は無傷で残る。

5.5. 難易度・活性化・再 genesis

Phase 3 はネットワークごとの活性化述語で制御される (testnet/mainnet では常時オン、devnet/simnet では決して有効化されず kHeavyHash のまま)。check_algo_id が header ごとに許可 id を厳密に強制し、単一アルゴリズム不変条件を保つ。アルゴリズム切替だけでは (PoW 免除の)genesis hash がバイト同一のまま残り、DB を消去せず再起動したノードが Phase-3 ブロックを Phase-2 履歴へ接ぎ木する危険がある。そこで MISAKA は Phase-3 の coinbase マーカ ("-bs3") で再 genesis を行い、稼働ネットワークの genesis hash を変更し、加えて記録された genesis が設定値と異なる DB の起動を拒否する startup genesis-mismatch ガードを備える。継承された bits は容易な launch フロアであり、DAA は un-throttle なマイニング下で最初の難易度窓内にハッシュレート均衡 $D \approx (\text{総 H/s})/\text{BPS}$ まで難易度を立ち上げ、停止しない。

5.6. 伝播安全性

高い BPS が安全なのは、GHOSTDAG の包含窓が伝播遅延を吸収する間に限られる。支配的な不等式は

$$\lambda \cdot D_{\max} \lesssim k, \quad (5)$$

であり、ブロックレート λ ・最悪伝播遅延 $D_{\max}$ ・クラスタパラメータ k を関係づける。これに違反すると DAG が分裂する。初期 testnet の事案では、地理的伝播遅延が Equation (5) を破ったことが DAG 分裂の原因と特定され、最小ブロック間隔の強制で解消された。EVM レーンは block にバイトを加えて $D_{\max}$ を押し上げるため、payload 込みの実測遅延で Equation (5) を再検証することが EVM 活性化の前提条件である (Section 7)。

6. DNS 確率的ファイナリティ・オーバーレイ

6.1. なぜ第二資源か

純 PoW は—単一ブロックの grinding に対し量子強化されていても—ハッシュパワーという単一資源に依存するため、PoW 多数派の攻撃者は原理的に任意の深さでリオーグできる。MISAKA は DNS 型 (Double-Nakamoto-Score 型) のファイナリティ・オーバーレイを加える。これはブロック生成とは分離された proof-of-stake の確認層であり、その価値は非代替性にある。履歴の置換には work と stake の双方の支配が必要であり、一方の資源の余剰は他方の不足を補わない。PoW/GHOSTDAG は引き続きブロック生成と tip 選択を担い不変であり、オーバーレイは on-chain の attestation オブジェクト群・決定論的 StakeScore・一つのリオーグ規則のみを加える。

6.2. オーバーレイが加えるもの

トランザクションが運ぶ 4 つのオブジェクト族、加えてパラメータ・読み出しビュー・規則である。

- **StakeBond**—コインを ML-DSA-87 バリデータ鍵にロックし、活性化 DAA score と、後述の長距離境界を満たす unbonding 期間を持つ。
- **StakeAttestation**—バリデータによる 1 つの ML-DSA-87 署名で、タプル (epoch, selected-chain anchor, validator-set commitment, bond outpoint) を対象とする。
- **StakeAttestationShard**—8–16 件の attestation を運ぶ block あたり上限付きトランザクション。複数 block にまたがる複数 shard が epoch 証明書を再構成するため、単一の巨大証明書トランザクションは不要である。
- **SlashingEvidence**—同一の (bond_outpoint, validator, epoch) からの 2 つの両立しない attestation。evidence 窓内に提出すると bond を焼却する。

6.3. 全アクティブ・ステーカー attestation

MISAKA は sortition 委員会を用いない。参加モデルは *stake* → *participate* であり、アクティブな stake bond を保有する全バリデータが毎 epoch attestation する。したがって epoch ごとの委員会部分集合はなく、sortition もなく、commit-reveal 乱数パイプラインもない。これは初期の委員会設計を覆すものであり、実装済みパイプラインの精査により委員会が block-validity の入力では一度もなかったことが確認された—各コンセンサス attestation 検証点は、attestation 自身が自己申告する validator-set commitment を含むメッセージ上の ML-DSA-87 署名を検証するため、sortition の除去はコンセンサス検査を除かずに機構のみを除く。10 BPS では attestation epoch(attestation_epoch_length_blue_score= 100) は壁時計で ≈ 10 s に過ぎず、数秒ごとにポーリングする単一バリデータでも追従できる。

6.4. 品質ゲート付き StakeScore(BFT-free)

素朴な StakeScore クレジットは線形かつ無ゲートである。epoch は $\text{credit}_e = (\text{included_stake}_e / \text{expected_stake}_e) \times \text{SCALE}$ を得るため、わずかな包含割合でも毎 epoch 正のクレジットを得て、十分長い窓では少数派ステークが score を累積しチェーンを確認へ押しやり得る。MISAKA は品質フロア φ_S を加える。これを下回る epoch は StakeScore をゼロしか得ない。これは意図的に BFT の三分の二多数決ではなく—PoS 層は BFT ファイナリティガジェットではない—少数派による長期累積を拒否する品質ゲートである。オーバーレイは BFT-free のままである。PoW/GHOSTDAG が唯一のブロック生成・tip 選択コンセンサスであり、StakeScore は第二の独立した確認次元に過ぎない。

6.5. 二次元リオーグゲート

候補フォークと canonical tip の共通祖先を I とし、 I 以降に累積した work-score と stake-score を $W(\cdot | I)$ 、 $S(\cdot | I)$ とする。最新の stake-confirmed anchor を含まない候補は、両次元で支配する場合にのみ受理される (Figure 3)。

$$(\text{cand} \not\geq \text{confirmed anchor}) \Rightarrow \left[W_{\text{cand}} > W_{\text{canon}} + m_W \wedge S_{\text{cand}} > S_{\text{canon}} + m_S \right], \quad (6)$$

さもなければリオーグは拒否される (DnsDominanceViolation)。マージン m_W, m_S はコンセンサスパラメータであり、両方を同時に上回ることが一方を上回るより指数的に起こりにくくなるよう設定される。実装上の注記を 2 点、率直に述べる。第一に、出荷される本番プリセットは確認の work-depth 閾値 $c_W = 0$ に設定されるため、確認述語は *stake-confirmed canonical lagged anchor* に縮退する。PoW 非代替性は (I 以降の差分を読む) リオーグゲート Equation (6) が強制するのであって、(累積 work を読む) 述語ではない。したがって本設計の正確な公的記述は「stake-confirmed canonical anchor + 二次元リオーグゲート」であり、「Double-Nakamoto confirmation」ではない。第二に、work-depth を真の確認入力にするには $c_W > 0$ と、累積 blue work から anchor 相対深度への変更の双方が必要である。

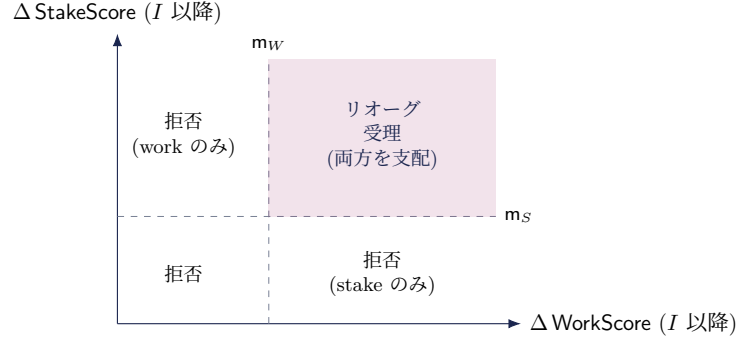


Figure 3. 二次元支配ゲート。stake-confirmed anchor を脱出する候補フォークは、累積 work と累積 stake の両方でマージン m_W, m_S だけ canonical chain を上回る右上領域でのみ受理される。いずれの軸も単独では十分でない。

6.6. 長距離境界・ロールアウト・パラメータ

PoS は PoW にない長距離面を導入するため、MISAKA は unbonding 期間に関するコンセンサス規則 $U \geq R + E$ で境界づける。ここで R は最大リオーグ horizon、 E は evidence 窓であり、bond が署名した equivocation がなお slash 可能な間は bond を引き出せない。オーバーレイは genesis では有効化できない (ステークが存在しない) ため、ロールアウトは三段階である。(i)PoW のみで bond/attestation トランザクションを拒否する *launch* 段階、(ii)bond と attestation を可視化のため受理するがゲートを強制しない *bootstrap* 段階、(iii) 総アクティブステークとアクティブバリデータ数が最小値を満たし活性化 DAA score に達すると起動し、以後 Equation (6) を強制する *activation* 段階である。本番プリセット (testnet/mainnet) は二次元確認と最小 stake bond 20,000,000 MSK (2×10^{15} sompi) を要求する。RPC `getDnsConfirmation` は `dnsConfirmed` と `lastDnsConfirmedAnchor` を報告し、指定 block hash の per-block ファイナリティにも応答できる。

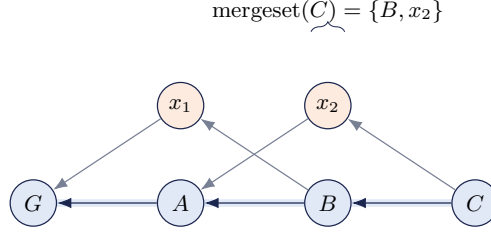
6.7. 運用上の堅牢化—バリデータとリモート signer

`kaspa-pq-validator` サイドカーはローカルノードに WebSocket-RPC で接続し、bond がアクティブな間 attestation する。鍵分離のため、`kaspa-pq-signer` はバリデータプロセスの外でバリデータ鍵を保持する独立デーモンであり、所有者専用 (0700) の Unix ドメインソケット越しに、設定可能ポリシ (permissive/audit-only/strict) ・strict モードのクラッシュ整合な signed-epoch ストアに支えられた anti-equivocation ガード・改竄検出可能なハッシュチェーン監査ログの下で署名要求に応える。これにより、侵害されたバリデータノードは鍵を持ち出すことも二重署名することもできない。

7. Selected-Parent EVM 実行レーン

7.1. 緊張関係と selected-parent 規則

外部ブリッジも L2 シーケンサも用いずに Ethereum EVM を L1 コンセンサスの一部として実行する。緊張関係は、EVM が本質的に逐次的なグローバル可変状態機械である一方、DAG は多数のブロックを並列に受理し仮想 selected chain を頻繁に再編成する点にある。受理された全ブロックの EVM トランザクションを仮想変更のたびに再実行すれば、恒常的な再順序付け・再実行を強いられる。MISAKA はこれを、DAG block B の EVM parent をその GHOSTDAG *selected parent*—直接 parent 集合全体でも現在の仮想 selected parent でもない—to 束縛するこ



selected chain(太線): $G \rightarrow A \rightarrow B \rightarrow C$. 青 = chain block、橙 = マージされた anticone。

$\text{ACCEPTEDEVMTXS}(C)$ は $\text{mergeset}(C)$ の payload を実行する。C 自身の payload は C の selected child を待つ。

Figure 4. BlockDAG 上の mergeset 遅延受理。chain block は自身の mergeset(selected parent とマージされた anticone) 内の EVM payload を実行し、自身の payload は 1 ステップ後に selected child が実行する。互いに素で被覆的な mergeset により全 payload はちょうど一度実行される。

とで解決する。

$$\begin{aligned} \text{EVM_PARENT}(B) &= \text{sp}(B), \\ \text{EVMRESULT}(B) &= \text{EXECUTE EVM} \left(\underbrace{\text{EVMSTATEROOT}(\text{sp}(B))}_{\text{parent 状態}}, \underbrace{B.\text{system_ops}}_{B\text{-scoped}}, \right. \\ &\quad \left. \underbrace{\text{ACCEPTEDEVMTXS}(B)}_{\text{mergeset}(B) \text{ より}}, \underbrace{\text{EVMBLOCKENV}(B)}_{\text{env, § 7.4}} \right). \end{aligned} \quad (7)$$

残りの設計を組織する単一の帰結は次である。**EvmResult(B)** は B の parents と B 自身の **system operation** のみの関数である。これは一度だけ計算され B の block hash の下に格納され、仮想リオーグは決して再実行しない。EVM ヘッドは canonical ポインタ (latest/ safe/ finalized) の切替で移動し、hot path に execute/revert は存在しない。

7.2. Mergeset 遅延受理 (off-by-one)

ユーザ EVM トランザクションの包含は全 DAG block に開かれている。実行は chain block の mergeset 受理として定義され、UTXO 受理を正確に写す。

$$\text{ACCEPTEDEVMTXS}(B) = \text{concat}_{X \in \text{sorted mergeset}(B)} (X.\text{evm_payload.transactions}(\text{payload 順})). \quad (8)$$

重要なのは、 B 自身のユーザ payload は $\text{EVMRESULT}(B)$ に含まれないことであり、それは B の selected child が受理する。これが off-by-one の原理的解決である。正当性は mergeset の互いに素性 (Equation (1)) に依拠する。selected chain 上で連続する mergeset は互いに素で過去を被覆するため、全 payload はちょうど一度実行される—二重実行も取りこぼしもない (Figure 4)。副次的利点として、 $\text{ACCEPTEDEVMTXS}(B)$ は B の parent 選択のみに依存するため、producer は自身の payload を選ぶ前に受理実行と commitment を終えられ、EVM 実行を PoW grinding と payload 選択から分離できる。

system operation(deposit claim, Section 8) は例外であり、 B 自身の payload に留まり B で実行される。これらは producer が選択し selected parent の UTXO view に対して検証されるためである。block 内の実行順は system operation が先、受理ユーザトランザクションが後である。

7.3. Commitment 構造とバージョンゲーティング

遅延受理では、block は二つを別個に commit する必要がある。すなわち自身の payload(データ)と、自身の mergeset を実行した結果 (実行) である。Borsh 上の keyed BLAKE2b-512 による

パラメータ	値	備考
EVM_HEADER_VERSION	2	genesis v0 / pre-EVM v1 は < 2 のまま
EVM_CHAIN_ID	0x4D534B(“MSK”)	EIP-155; 公開 Ethereum net と非衝突
EVM fork	revm SpecId::SHANGHAI	Cancun は後続 fork; 自動追従なし
EVM_NATIVE_SCALE	10^{10}	sompi(8 桁) → wei(18 桁)
EIP-1559	elasticity 2, denom 8, basefee burn	初期 basefee 1 gwei
WMISAKA predeploy	0x...F001	WETH9 互換 wrapped native
Withdraw precompile	0x...F002	Section 8
ML-DSA-87 verify precompile	0x...F003	FSL 専用 (Section 10)
実行 gas 上限	G_{limit} / chain block	per-second 導出 (§ 7.6)

Table 3. 凍結された EVM レーンパラメータ。chain id “MSK” は公開 Ethereum ネットワークと非衝突に選ばれ、mainnet 最終値は launch 時に確定する。

64 バイト header フィールド 2 つがこれを担う。

$$\begin{aligned} \text{evm_payload_hash}(B) &= \text{BLAKE2b-512}(\text{key} = \text{EvmPayload64}, \text{Borsh}(B.\text{evm_payload})), \\ \text{evm_commitment_root}(B) &= \text{BLAKE2b-512}(\text{key} = \text{EvmCommitment64}, \text{Borsh}(\text{EVMEXECUTIONHEADER}(B))). \end{aligned} \quad (9)$$

実行ヘッダは EVM の state/transactions/receipts root ・ gas used ・ base fee ・ EVM ブロック番号と timestamp ・ coinbase ・ 受理および skip したトランザクション数 ・ 供給不変条件用の $O(1)$ total-native-balance アキュムレータ ・ base-fee burn アキュムレータを保持する。二層ハッシュ規約が正式仕様である。EVM 内部 root は keccak-256/32 バイト、L1 コンセンサス commitment は Hash64/keyed BLAKE2b-512 である。

header フィールドは常に存在する (既定はゼロ) が、header-hash の preimage に入るのは $\text{header.version} \geq \text{EVM_HEADER_VERSION} = 2$ のときのみである。genesis は version 0、pre-EVM のマイニング済みブロックは version 1 でいずれも < 2 であるため、すべての genesis hash と pre-EVM ブロック識別子はバイト単位で不変である—EVM レーンは履歴を乱さないバージョンゲート付き hard fork である。

7.4. EVM 実行環境

Table 3 に凍結パラメータを挙げる。初期 fork は revm の SpecId::SHANGHAI(PUSH0 と現行 Solidity 出力、Uniswap v2/v3 が動く) であり、Cancun/EIP-1153 の transient storage は後日別個に活性化する fork で、上流への自動追従はしない (fork bump は hard fork)。EVM ブロック番号は selected chain に沿って増加する ($\text{evm_number}(B) = \text{evm_number}(\text{sp}(B)) + 1$)。EVM 論理時刻を BPS に結合させないため、timestamp clamp は厳密単調ではなく非減少とする。

$$\text{evm_timestamp_sec}(B) = \max(\text{header_ts}(B), \text{evm_timestamp_sec}(\text{sp}(B))),$$

よって連続する複数 EVM ブロックが同一 timestamp を共有し得る。開発者には sequencing には `block.number` を、`block.timestamp` は低解像度の壁時計としてのみ用いるよう案内し、これにより Uniswap v2/v3 の TWAP オラクルは正しく動作する (等値 timestamp で skip または early-return する)。循環依存—header hash は EVM 結果を commit するため EVM 入力にできない—を避けるため、`prevrandao` と `blockhash` は selected-parent の系譜のみから導出する。

7.5. Validity と 5 分類 skip セマンティクス

遅延受理では受理 producer が payload 内容を選んでいないため、payload の欠陥が受理 block を無効化することは許されない。MISAKA はトランザクション結果を 5 クラスに分類する (Table 4)。要点は次のとおり。*payload-admission* 違反 (クラス 1) は安価な構文検査で、その *payload block*

クラス	条件	扱い	帰責
1	符号化/署名不正、chain-id 不一致、固定 intrinsic フロア未満、サイズ超過	payload block 無効 (body validation)	payload producer
2	nonce 不正、残高 < 前払コスト、max-fee < basefee(B)	決定論的 skip; receipt なし; nonce 不変	なし
3	重複 tx hash	クラス 2 nonce 規則で自動 skip	—
4	revert・OOG・precompile/slippage/hook 失敗	実行; receipt status 0; gas 課金	ユーザ
5	受理 gas 上限超過 (canonical 末尾)	決定論的 prefix skip; nonce 不変	—

Table 4. 5 分類 skip セマンティクス。block を無効化し得るのはクラス 1(payload block 自身の構文欠陥) と commitment/system/UTXO 検査のみ。他は EVM 結果に不可視な skip。

を body validation で無効化する (その producer の責)。受理時 skip (クラス 2: nonce 不正、残高不足、max-fee が block の basefee 未満) は receipt なし・gas 課金なし・nonce 不変の決定論的 skip で、payload 作成時に basefee が未知だったため誰の責でもない。重複 (クラス 3) はクラス 2 の nonce 規則で自動吸収され、seen-set は不要である。実行時失敗 (クラス 4: revert・OOG・slippage) は実行され、status 0 の receipt と gas 課金を伴う (ユーザの責)。gas 上限超過 (クラス 5) は決定論的 prefix skip である。クラス 2/3/5 は EVM 結果に不可視であり—state・receipts・gas used に影響しない—nonce 不変の skip されたクラス 2/5 トランザクションは、条件が満たされれば後続 block で自然に再受理される。block が無効または chain 失格になるのは、commitment/root 不一致、system-op のコンセンサス違反、UTXO-diff 不整合、または自身 payload のクラス 1 構文違反のときのみである。

7.6. Gas 予算と二段階上限

per-DAG-block の payload バイト上限だけでは、大きな mergeset をマージした chain block の実行予算が無制限になり得るため、MISAKA は二つの上限を用いる。MAX_EVM_PAYLOAD_BYTES_PER_DAG_BLOCK (包含側、body validation で検査) と MAX_EVM_ACCEPTED_GAS_PER_CHAIN_BLOCK (実行側、per-second 導出の gas 上限に一致) である。実行側上限は gas_limit に対する決定論的 prefix-take として適用される。ACCEPTED_EVM_TXS(B) を canonical 順に走査し、累積 gas limit が上限を超える最初のトランザクション以降をすべてクラス 5 skip する。gas_limit 基準で判定することで受理集合が実行前に確定し、並列スケジューラへの入力が固定される。carry queue は用いない (carry は queue root を commit 状態に変える必要がある)。skip されたトランザクションは nonce 不変のため再包含で回復される。

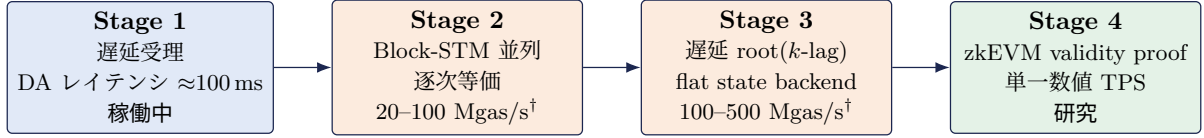
7.7. 手数料モデル

basefee は burn される (監査のため累積)。各トランザクションの priority fee は、それを包含した payload block が宣言する evm_coinbase へ credit される—包含は payload マイナーが提供する DAG 全幅の希少資源である一方、実行は chain 候補性の必須条件であり受理 producer に追加インセンティブは不要だからである。COINBASE opcode は受理 chain block の coinbase を返す (1 block 1 coinbase)。したがって priority fee の受益者は EVM block 内でトランザクションごとに異なり得る。これは EVM 実行環境差として開示され、トランザクションごとの受益者は misaka_feeRecipient receipt フィールドで公開され、eth_getBlockByNumber.miner は受理 coinbase を返す。

7.8. 段階的スケーラビリティ

EVM レーンには 4 段階のロードマップ (Figure 5) があり、単一数値の TPS 競争は最終段階のみに属するという明示的方針を持つ。

Stage 1 (稼働中、v0.4 fork)。EVM 包含を全 DAG block に開放し、実行は mergeset 遅延受理とする。包含レイテンシは DAG block 間隔 (10 BPS で ≈ 100 ms)。



† 技術目標値、低～中コンテンション; Section 7.11 の主張規律に従う。

Figure 5. EVM スケーラビリティの4段階ロードマップ。Stage 1 は稼働中、Stage 2-3 は実装/fork 作業、Stage 4 は研究。スループット数値は仕様保証値ではなく技術目標値である。

Stage 2(実装、コンセンサス変更なし)。 GHOSTDAG 順序を固定したまま optimistic 並列実行 (Block-STM 系) を行う。唯一の仕様要求は bit-exact な逐次等価性であり、並列結果は canonical 順の逐次実行と一致しなければならない。EIP-2930 access list はスケジューラ ヒントとしてのみ用いる (gas 割引なし)。期待される高速化は workload 依存で、低コンテンションで 10-20×、単一 hot pool では伸びない。

Stage 3(v0.5 fork-gated)。 state-root commitment の遅延。実行は引き続き chain-context validation 時に行うが、merkle 化と commitment を k block 遅延させ ($\text{state_root}(B) = \text{state_root}(\text{ANCESTOR}(B, k))$)、merkle 化をクリティカルパスから外す。state は非同期 merkle 化を伴う flat key-value ストアへ移行し、ノード状態は $\text{canonical_tip} \cdot \text{executed_tip} \cdot \text{verified_tip} = \text{canonical_tip} - k$ を区別し、misaka_verified RPC タグと mismatch ロールバック/失格プロトコルを伴う。達成可能な gas/秒は $G_{\text{sec}}^{\text{max}} = \text{REPLAY_SPEED} / \text{SAFETY_MARGIN}(10-20\times)$ に拘束される。

Stage 4(研究)。 全ノード再実行を proof 検証に置換する enshrined validity-proof(zkEVM) レーン。単一数値の TPS 競争への唯一の経路であり、目標は proof レイテンシ $\leq 10\text{s}$ 、128 ビット安全性、proof サイズ $\leq 300\text{KiB}$ で、prover liveness・分散性・データ可用性・強制包含の要件を規定する。

7.9. PQ-only との整合と secp 隔離

EVM レーンは secp256k1/ECDSA(ecrecover と署名検証) を独立署名ドメインとして再導入する。PQ-only 不変条件との整合は構造的に強制される。EVM の型 (consensus-core) は常にコンパイルされ secp-free であり、EVM の executor(revm と k256 依存) は cargo evm feature(既定オフ) の背後にあり、既定 kaspad は secp-free のままで (CI ガードが本番バイナリ群で強制)、EVM-active ネットワークに非 EVM バイナリで参加すると silent fork ではなく明示的に停止する。EVM レーンの量子リスクは Ethereum と同等と開示され、耐量子保証は UTXO レーンのみに及ぶ。

7.10. コンセンサス不変条件

本レーンは 10 個の不変条件で規定される (Table 5)。これらは適合実装が保つべき契約であり、適合テストの基礎である。

7.11. 性能主張の規律

MISAKA は仕様保証値 (コンセンサスパラメータから導出可能) と技術目標値 (ベンチマーク裏付け、条件付きで引用) を分離し、単一 hot object の数値を一般スループットとして広告することを禁じる。レイテンシは常に「100ms」という単一見出しではなく 3 層で報告する (Table 6)。data-availability 包含 ($\approx$ DAG block 間隔)、実行 receipt ($\approx$ selected-chain 間隔)、確率的ファイナリティ (blue-work 確認深度、safe/ finalized として公開) である。他の並列システムに対する位置づけも限定する。UTXO レーンは owned-object/fastpath 相当、EVM レーンは shared-state/consensus-path 相当であり、全ノード再実行 EVM について単一数値の王座は主張しない。文脈として、1-3 Mgas/s 設定での送金スループットは概ね 60-140 TPS、並列実行と flat state backend で 5,000-24,000 TPS へ上昇する。これらは上記の意味での技術目標値であり、10 種の workload ベンチマーク行列 (native transfer、独立残高の ERC-20、hot-token 独立残高、単

不変条件	
I1	mergeset 分割: 各 EVM ユーザ tx は selected chain ごとに高々一度実行される。
I2	EVMRESULT(B) は B の parents と B の system ops のみに依存し、ユーザ payload に依存しない。
I3	仮想変更はポインタ更新のみ (再実行なし)。
I4	skip(クラス 2/3/5) は state・receipts・gas used・nonce に影響しない。
I5	並列実行は canonical 順の逐次実行と bit 単位で一致する。
I6	priority fee 移転 + burn + 残高変化は供給不変条件を満たす。
I7	引出しの synthetic UTXO は受理 block 自身の UTXO diff にのみ現れる。
I8	evm_timestamp_sec は selected chain 上で非減少である。
I9	現在の L1 / EVM block hash は EVM 入力にならない。
I10	既定ノードビルド (EVM feature オフ) は secp256k1 をリンクしない。

Table 5. EVM レーンのコンセンサス不変条件。

量	区分	値	条件
DA 包含レイテンシ	仕様保証	$\approx$ DAG block 間隔	BPS 依存
receipt レイテンシ	仕様保証	$\approx$ selected-chain 間隔	実効幅依存
Stage 2 スループット	技術目標	20–100 Mgas/s	低～中コンテンション
Stage 3 スループット	技術目標	100–500 Mgas/s	flat backend, NVMe
送金 TPS	技術目標	gas/s $\div$ 21,000	多数の独立 sender
Uniswap swap TPS	技術目標	gas/s $\div$ 120–150k	多数 pool 分散時のみ

Table 6. 性能主張行列。仕様保証値はコンセンサスパラメータから従い、技術目標値はベンチマーク裏付けで条件付き。

一/多数 Uniswap pool、集中流動性、清算ホットスポット、依存バンドル、敵対的 skip flood、anchor から tip への replay) に対して検証される。

8. UTXO $\leftrightarrow$ EVM ブリッジ

二つのレーンは一つのネイティブコインを共有する。価値は、単一供給を保存する in-consensus の deposit/withdraw 機構—外部カストディアンも wrapped IOU もなし—を通じてレーン間を移動する (Figure 6)。

8.1. 供給不変条件

常時、

$$\text{UTXO_balances} + \text{EVM_deposit_locks} + \text{EVM_native_balances} + \text{burned} = \text{issued}. \quad (10)$$

Equation (10) は実行ヘッダ (Section 7) の evm_total_native_balance アキュムレータから $O(1)$ で検査される。priority fee 移転・basefee burn・残高変化はいずれもこれを保存する (不変条件 I6)。

8.2. Deposit(二段階)

1. **Lock**. ユーザは UTXO レーンで EVM_DEPOSIT_LOCK 出力 (subnetwork 0x20、script class EvmDepositLock) を作り、宛先 EVM アドレス・timeout_daa_score・claim_tip_sompi 包含インセンティブを持たせる。
2. **Claim**. producer は DepositClaim system operation を自 block B の payload に入れる。claim は B の selected-parent の UTXO view に対して検証される (同一 block 内で作られた lock は未だ claim 不可)。成功時は宛先に amount_sompi $\times$ EVM_NATIVE_SCALE を credit し、 B 自身の per-block UTXO diff で lock を消費する。
3. **Refund**. timeout 後、ユーザは通常 spend で lock を回収する。

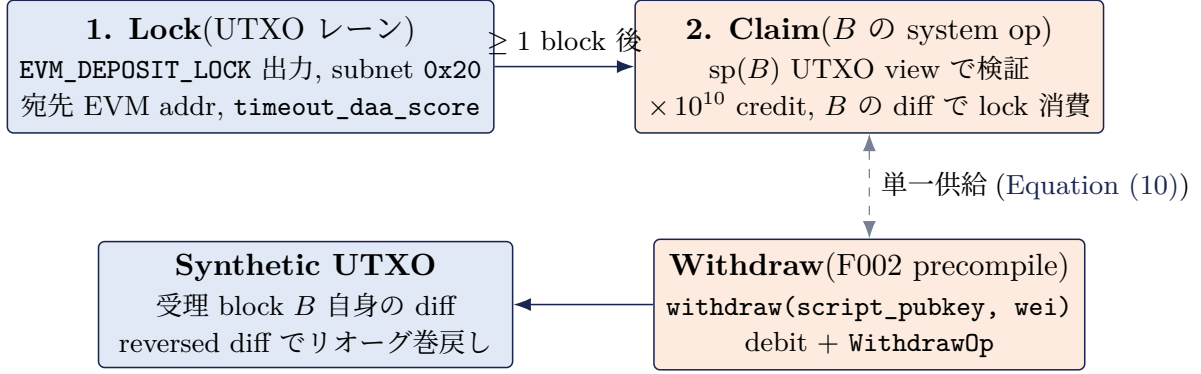


Figure 6. UTXO ↔ EVM ブリッジ。deposit は排他的 refund 窓を持つ二段階の lock → claim、withdraw は EVM 残高を burn し受理 block 自身の diff に synthetic UTXO を materialize するため、リオーグはそれらを実行して巻き戻す。

排他窓—claim valid $\iff$ accepting_block_daa_score < timeout_daa_score—により、claim と refund が同時に有効になる DAA 領域は存在しない。claim は EVM block あたり有界である (最大 256 件、payload 64 KiB、25,000 gas/claim の system-gas 上限)。違反は block を失格させる。

8.3. Withdraw(F002 precompile)

受理ユーザトランザクションは `0x...F002` の precompile `withdraw(bytes script_public_key, uint256 amount_wei)` を呼ぶ。ユーザ入力起因の不具合—calldata 不正・amount 0・amount_wei が EVM_NATIVE_SCALE の正確な倍数でない・script 規則違反・残高不足—はクラス 4 として revert し、block は valid のまま残る。成功時は EVM 残高を debit し WithdrawOp を emit する。引き出すトランザクションの payload は別 block 由来だが、実行は受理 block B で行われるため、synthetic UTXO 出力は B 自身の per-block UTXO diff に materialize される (不変条件 I7)。その outpoint は凍結ドメインから決定論的に導出され、同一 block 内では unspendable である。per-block diff に載るため、リオーグ時の巻き戻しは既存の reversed-diff 経路で自動的に行われる。

9. ネイティブコインと発行

ネイティブコインは MSK であり、sompi 建てである ($1 \text{ MSK} = 10^8 \text{ sompi}$)。供給上限は 300 億 MSK—150 億のプリマインに 20 年間で 150 億のネットワーク発行—である。

プリマイン。 プリマインは単一の ML-DSA-87 P2PKH 鍵への 150 億 MSK 出力 1 つで、genesis の UTXO commitment が commit する。mainnet ではプリマイン所有者は、カストディ儀式が実鍵を確定するまでプレースホルダ (全ゼロの unspendable payload) である。その儀式は mainnet hash を再 genesis し、launch-safety フラグが儀式 pending を示す。テストネットは誰でもテスト資金を用意できるよう公開回収可能なプリマイン鍵を用いる。

発行スケジュール。 ネットワーク発行は 20 年間で年率 5% 減衰し、その後停止する。 E_y を y 年目の発行量として、

$$E_y = E_1 \cdot 0.95^{y-1}, \quad E_1 = 15 \times 10^9 \cdot \frac{1 - 0.95}{1 - 0.95^{20}} \approx 1.169 \times 10^9 \text{ MSK/年}. \quad (11)$$

per-second サブシディは事前計算された 241 項の月次テーブルであり、coinbase マネージャが各項を実際の BPS で割る。10 BPS では 1 年目のレートは $\approx 3.70468 \text{ MSK/block}$ (BPS 除算前で $3,704,683,450 \text{ sompi/s}$)、年次比は 10^{-4} 以内に 0.95、発行は月 240 以降ちょうどゼロで、20 年間で総計 ≈ 150 億 MSK となる (Table 7)。

年	10 BPS での block あたりサブシディ (MSK)	年間発行 (MSK)
1	3.70468	$\approx 1.169 \times 10^9$
2	3.51945	$\approx 1.111 \times 10^9$
5	3.01749	$\approx 9.52 \times 10^8$
10	2.33487	$\approx 7.37 \times 10^8$
20	1.39798	$\approx 4.41 \times 10^8$
> 20	0	0

Table 7. ネイティブコイン発行 (年率 5% 指数減衰、20 年)、Equation (11) より導出。表中は代表点であり、発行は 20 年目以降に終了する。

報酬の振り分け。 block 報酬はファイナリティ・オーバーレイの包含経済の下で attestation するバリデータへ fan-out される (サブシディの worker-inclusion サブプールが attestation 包含を賄う)。coinbase fan-out 機構を再利用しつつ、額のモデルは Section 6 の品質ゲート設計に従う。EVM レーンでは basefee が burn され、レーン利用量に比例して発行を流通から除去する。

10. Fact Settlement Layer(ロードマップ)

Fact Settlement Layer(FSL) は計画中のサブシステム (設計 v0.3、経済 v0.5) であり、未実装で、コンセンサス変更は EVM precompile 一つのみである。Table 3 で予約した 0x...F003 ML-DSA-87 verify precompile を動機づけるため、ここで要約する。

FSL は意図的に予測市場ではなく、機械検証可能な事実を決済する evidence-anchored 機構である。4 つの原則が駆動する。第一に曖昧性を裁定時でなく作成時に消す。claim は Predicate DSL で記述され、その predicate_hash は作成時に固定され post-hoc 編集経路がないため、「イベント時刻 対 確認時刻」型の紛争は構造的に排除される。第二に裁定階層を機構化する—証拠 > 評判 > 専門性 > ステーク > 投票—エスカレーションラダーとして実装し、証拠を引用しない票は無効で、void(裁定不能) エスケープハッチを常設する。ラダーは 5 段である。L0 自動ソース解決 (template が許す場合のみ、必須の challenge 窓付き)、L1 optimistic 解決 (bond 付き propose、challenge 窓)、L2 パネル評決、L3 ステーク加重の court 評決と控訴 (最終手段)、L4 void であり、状態遷移は Created → Proposed → Settled で、Disputed → PanelVerdict → CourtVerdict または Voided へ分岐する。第三に検証可能な説明責任。評決は第三者が再計算可能な forensic evidence package を伴い (parser id/version と抽出フィールドを持ち、本体は pin 義務 + storage challenge + slashing の下で archival ネットワークに格納)、参加は entity-bound(一エンティティ一席) で、経済的保証は単一の保証額ではなく分解されたリスクとして公開される。第四に主張ではなく実績で信用を得る。決済権を持たない Evidence Graph API をまず出荷し、方法論を固定した公開 Shadow Resolution の実績を作り、その上に Settlement Adapter を載せる。

FSL は裁定パネルと陪審選出ビーコン (attestor commit-reveal ビーコン、決して prevrandao ではない) に DNS ファイナリティ・オーバーレイを再利用し、証拠・評決の帰属の PQ-safe ドメインとして ML-DSA-87 と Hash64 を用いる。ソース評判スコアは authoritative ではなく informative である (初期値の例: official 980、exchange 950、news agency 900、registered 400、anonymous 100)。これらは L0 の許容判定・bond 割引・表示にのみ用い、court は証拠を de novo で判断する。経済設計は新規トークンを追加しない。設計が率直に述べる正直な限界は、オラクル問題は解けず構造化できるのみ、ということである。FSL は形而上的真実ではなく「プロトコルが証拠の足跡とともに裁定したもの」を決済する。

11. 実装とネットワーク状況

MISAKA は Rust 版 Kaspas フルノードである *rusty-kaspa* のフォークである。ノードバイナリは kaspad の名を保ち、crate も上流の kaspa-* 名を保つ—これはフォークであって改名ではない—一方でネットワーク・アドレス (misaka/misakatest/misakadev)・ブランディングは MISAKA のものである。上流と同じ ISC ライセンスで配布され、上流のすべての功績は Kaspas プロジェクトに帰する。

稼働ネットワークは実験的な testnet-10 であり、block エクスプローラは misakascan.com、DNS シーダは seeder1.misakascan.com / seeder2.misakascan.com、P2P ポートは 26211(mainnet 26111、devnet 26611) である。PQ-only コンセンサスと DNS ファイナリティ報酬オーバーレイは、定義済み全ネットワークで genesis から有効 (pq_activation_daa_score = 0、dns_activation_daa_score = 0) であり、testnet/mainnet パラメータ集合はさらに本番ファイナリティポリシ (二次元確認、20M-MSK 最小 bond) を強制する。EVM レーンは 2026-06-11 に、testnet genesis hash を不変 (cf4c48fe...) に保つ 3 ホスト再 genesis カットオーバーで testnet 上で活性化された。活性化後の header は EVM_HEADER_VERSION 2 を持つため、pre-EVM ノードと旧チェーンは双方向にバージョン隔離され、EVM トランザクション中継はプロトコルバージョン引き上げをまたいで後方互換である。mainnet パラメータ集合は定義済みだが **launch も本番承認もされていない**。

ツリーはノード・マイナー・バリデータサイドカーとそのリモート signer・DNS シーダ・ブリッジ・PQ コマンドラインウォレット・WASM SDK・ブラウザ拡張ウォレットを同梱する。CI ガードは consensus crate やノードが secp256k1 をリンクすると hard-fail し、EVM executor は evm ビルド feature 経由でのみ到達可能である。

12. セキュリティ考察

耐量子主張の範囲。 PQ 保証はトランザクション認可 (ML-DSA-87)・コンセンサス識別子 (Hash64)・UTXO アキュムレータ (格子 LtHash32) に及ぶ。トランスポートの秘匿性には及ばず、ML-KEM ハイブリッド鍵交換を有効にした場合のみ耐量子である—launch 時スコープ外でありその旨開示する。旧来 Kaspero アドレスは耐量子ではなく、本設計では構造的に表現不能である。

EVM レーンは耐量子ではない。 EVM 署名ドメインは secp256k1 で、量子脅威への露出は Ethereum と同等である。これは開示され、耐量子保証は明示的に UTXO レーンのものである。本レーンはビルド feature の背後に隔離され既定ノードは secp256k1 を決してリンクせず、非 EVM バイナリは EVM-active ネットワークで silent fork せず停止する。

compute-only PoW 下のリオーグ耐性。 Phase-3 PoW は ASIC 平等的でないため、単一資源 (ハッシュパワー) 支配がメモリ困難 PoW より起こりやすい。緩和は DNS オーバーレイである。stake-confirmed anchor を脱出する深いリオーグは Equation (6) の二次元支配ゲートを通過せねばならず、純 PoW 多数派は stake も支配しない限り通過できない。品質フロア φ_S は少数派ステークによる長期累積を拒否する。

長距離と equivocation。 unbonding 境界 $U \geq R + E$ は、bond が署名した equivocation が提出可能な間は bond を slash 可能に保つ。slashing-evidence オブジェクトは evidence 窓内に二重署名 bond を焼却し、リモート signer の strict ポリシはバリデータプロセス外にクラッシュ整合な anti-equivocation ガードを加える。

サービス妨害と伝播。 ML-DSA オブジェクトは大きいため、署名検証を価格付けし block 計算を有界化するよう mass を再校正し、EVM レーンのバイト・gas 予算は UTXO mass 予算と分離されるため、満載の EVM payload も UTXO スループットを低下させない。EVM payload は block を肥大させるため、EVM 活性化は payload 込みの実測遅延で伝播安全不等式 Equation (5) を再検証することに条件づけられる。

依存の姿勢。 署名依存の耐量子安全性は静的ではない。ビルドは libcrux-ml-dsa をピン留めし、CI で advisory と audit 検査を走らせ、格子ライブラリを固定仮定ではなく監視対象面として扱う。

13. 関連研究

MISAKA の DAG エンジン は Kaspas の GHOSTDAG [1] であり、そこからブロック生成・blue-work tip 選択・難易度調整アルゴリズムを継承する。本書の寄与は DAG プロトコルそのものではなく、耐量子移行・ファイナリティ・オーバーレイ・EVM レーンである。署名・ハッシュ・KEM の標準は NIST 耐量子スイートである。署名に ML-DSA(FIPS 204)[2]、ハイブリッドトランスポートの選択肢に ML-KEM(FIPS 203)[3]、ステートレスハッシュの代替に SLH-DSA(FIPS 205)[4] を用い、対称原始関数は BLAKE2 [6] と SHA-3 [5] である。格子 UTXO アキュムレータは加法準同型 LtHash の系譜 [7] に従う。

EVM レーンの実行モデルは並列実行と遅延状態の文献に学ぶ。固定 preset 順序の Block-STM optimistic 並行性 [8]、および遅延 Merkle root を伴う Monad 型非同期実行 [9] を、実行は引き続き chain-context validation 時に行い commitment のみ遅延させるよう適応する。性能の位置づけは Sui の owned 対 shared object 実行経路 [10] に対して述べ、再実行 EVM の単一数値比較を意図的に避け、その競争を zk-rollup [11] 的な将来の validity-proof レーンとリアルタイム L1 zkEVM proving 目標に留保する。EVM 環境の規約は chain-id replay 保護・optional access list・手数料市場の Ethereum 標準 [12, 13, 14] に従う。Fact Settlement Layer の evidence-anchored 裁定としての枠組みは、optimistic なトークン投票オラクルの文書化された失敗モードへの応答である。

14. 結論と今後の課題

MISAKA は、構成要素が単に共存するのではなく互いを補強するよう選ばれている限り、単一の基盤層がコンセンサス表層で耐量子であり、高スループットの BlockDAG を保持し、consensus-native な EVM をホストできることを示す。トランザクション認可を ML-DSA-87、コンセンサス識別子を 64 バイトの keyed BLAKE2b-512、UTXO アキュムレータを格子 LtHash とすることで、UTXO レーンの量子マージン非対称が解消される。二資源の DNS ファイナリティ・オーバーレイを加えることが、高速な compute-only proof-of-work を許容可能にする。深い履歴の書き換えには work に加えて stake の支配が必要になるからである。そして EVM parent を GHOSTDAG selected parent に、Kaspa 準拠の mergeset 遅延受理とともに束縛することで、本質的に逐次的な EVM が再編成する DAG 上で生き、「block hash に対して不変、リオーグはポイント切替」を真に保つ—外部ブリッジもシーケンサもなしに。

今後の課題は段階的ロードマップに従う。逐次等価な並列実行 (Stage 2)、flat backend を伴う遅延 state-root commitment (Stage 3)、単一数値スループットへの唯一の誠実な経路としての研究段階 enshrined validity-proof レーン (Stage 4)、Fact Settlement Layer とその ML-DSA-87 verify precompile、そして暗号面では耐量子主張を秘匿性へ拡張する任意の ML-KEM ハイブリッドトランスポートである。全体を通じて、本プロジェクトはコンセンサスパラメータが保証するものとベンチマークが目標とするものを分離する明示的規律を保つ。MISAKA は Kaspa プロジェクトの BlockDAG の上に築かれており、上流のすべての功績は Kaspa に帰する。

References

- [1] Y. Sompolinsky, S. Wyborski, A. Zohar. *PHANTOM GHOSTDAG: A Scalable Generalization of Nakamoto Consensus*. Cryptology ePrint Archive, Report 2018/104; および Kaspa プロジェクト文書 wiki.kaspa.org.
- [2] National Institute of Standards and Technology. *FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA)*. 2024.
- [3] National Institute of Standards and Technology. *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM)*. 2024.
- [4] National Institute of Standards and Technology. *FIPS 205: Stateless Hash-Based Digital Signature Standard (SLH-DSA)*. 2024.
- [5] National Institute of Standards and Technology. *FIPS 202: SHA-3 Standard*. 2015.

- [6] M.-J. Saarinen, J.-P. Aumasson. *The BLAKE2 Cryptographic Hash and MAC*. RFC 7693, 2015.
- [7] Meta / R. Lewi et al. *Securing Update Propagation with Homomorphic Hashing*(LtHash). Cryptology ePrint Archive, Report 2019/227.
- [8] R. Gelashvili et al. *Block-STM: Scaling Blockchain Execution by Turning Ordering Curse to a Performance Blessing*. arXiv:2203.06871, 2022.
- [9] Monad Labs. *Asynchronous Execution and Deferred Merkle-Root Commitment*. Monad 文書 docs.monad.xyz.
- [10] Mysten Labs. *Sui: Owned- and Shared-Object Execution Paths*. Sui 文書および性能レポート docs.sui.io.
- [11] Ethereum Foundation. *Zero-Knowledge Rollups および Shipping an L1 zkEVM: Realtime Proving*. ethereum.org, 2025.
- [12] V. Buterin. *EIP-155: Simple Replay Attack Protection*. Ethereum Improvement Proposals, 2016.
- [13] V. Buterin, M. Swende. *EIP-2930: Optional Access Lists*. Ethereum Improvement Proposals, 2020.
- [14] V. Buterin et al. *EIP-1559: Fee Market Change for ETH 1.0*. Ethereum Improvement Proposals, 2019.